

A Formal Mathematical Treatment of Node-Link Graphs: Verification of In-Place Reversals and Constant-Space Isomorphic Deep Copying

Abstract—This paper presents a formal mathematical model of singly and doubly linked list data structures, establishing rigorous operational semantics for node-link graphs. We address the core algorithms of search, index-based retrieval, and structural node mutation. Specifically, we present a formal verification of the linear-time, in-place list reversal algorithm using loop invariants and axiomatic state-transition semantics. Furthermore, we analyze the problem of duplicating linear structures with arbitrary transverse bridges (random pointers). We compare the standard hash-map-based $O(N)$ auxiliary space deep-copy approach with an optimal, interleaved $O(1)$ auxiliary space algorithm, proving the latter’s structural and spatial optimality. Finally, we formalize function activation records, stack frame state updates, and heap-allocation topologies to bridge abstract algebraic algorithms with hardware-level memory layouts.



1 Introduction

Linear node-link structures, colloquially referred to as linked lists, serve as fundamental primitives in computer science. Unlike contiguous memory buffers such as arrays, linked lists organize information as distributed memory nodes interconnected via explicit pointer addresses. While this representation allows for $O(1)$ -time dynamic structural modifications, it introduces complex challenges in algorithmic verification, pointer safety, and spatial complexity bounds.

This paper provides a rigorous mathematical treatment of linear node-link structures. We define the algebraic properties of node-link graphs and establish formal axiomatic semantics for standard operations including search and indexed retrieval. We then focus on two primary algorithmic challenges:

- 1) **In-Place Pointer Reversal:** We provide a formal proof of correctness for the classic iterative list-reversal algorithm, using loop invariants over state transition systems.
- 2) **Isomorphic Graph Deep-Copying under Strict Space Constraints:** We analyze the deep copying of linked lists augmented with arbitrary transverse bridges (random pointers). We present a formal comparison between the classical $O(N)$ -space hash-map implementation and the optimal $O(1)$ -auxiliary space interleaved-node algorithm, establishing the space optimality of the latter.

Finally, we bridge these abstract algorithms to concrete execution systems by formalizing the memory topology of the program stack and heap.

2 Formal Model of Node-Link Graphs and Basic Operations

Let $\mathcal{A}$ be the set of valid memory addresses, and let $\mathcal{V}$ be the value domain of the data stored within each node. A Node u is modeled as a tuple:

$$u = \langle d_u, n_u, r_u \rangle \in \mathcal{V} \times (\mathcal{A} \cup \{\emptyset\}) \times (\mathcal{A} \cup \{\emptyset\})$$

where $d_u \in \mathcal{V}$ represents the payload data, $n_u \in \mathcal{A} \cup \{\emptyset\}$ represents the primary successor pointer (next), and $r_u \in \mathcal{A} \cup \{\emptyset\}$ represents an auxiliary transverse pointer (random).

A singly linked list is represented as a directed graph $G = (V, E)$, where $V = \{u_1, u_2, \dots, u_N\}$ is the set of node-tuples, and E is the set of primary directed edges defined by successor pointers:

$$E = \{(u_i, u_{i+1}) \mid n_{u_i} = \text{Addr}(u_{i+1})\}$$

where $\text{Addr} : V \rightarrow \mathcal{A}$ is an injective mapping, and $n_{u_N} = \emptyset$.

2.1 Linear Search Semantics

The search operation determines whether a target value $x \in \mathcal{V}$ exists within the node payloads. Let $\text{Search}(H, x)$ be a function mapping a list head pointer $H \in \mathcal{A}$ and a target value x to a boolean output $\mathbb{B} = \{0, 1\}$. The operational semantics of the search algorithm are defined recursively:

$$\text{Search}(p, x) = \begin{cases} 0 & \text{if } p = \emptyset \\ 1 & \text{if } p \neq \emptyset \wedge d_p = x \\ \text{Search}(n_p, x) & \text{if } p \neq \emptyset \wedge d_p \neq x \end{cases}$$

where d_p and n_p denote the data and next pointer of the node at address p , respectively.

For a finite list of length N , the algorithm terminates in at most N steps, yielding a worst-case time complexity of $O(N)$ and an auxiliary space complexity of $O(1)$ when implemented iteratively.

2.2 Indexed Retrieval

Given an index $K \in \mathbb{N}_0$, retrieval of the K -th node’s address is modeled by the operator $\mathcal{R} : \mathcal{A} \times \mathbb{N}_0 \rightarrow \mathcal{A} \cup \{\emptyset\}$:

$$\mathcal{R}(p, K) = \begin{cases} p & \text{if } K = 0 \\ \emptyset & \text{if } p = \emptyset \wedge K > 0 \\ \mathcal{R}(n_p, K - 1) & \text{if } p \neq \emptyset \wedge K > 0 \end{cases}$$

This operator establishes that node access is strictly sequential, requiring $O(K)$ pointer traversals.

2.3 Structural Mutation: Insertion

Inserting a new node u_{new} at index i requires structural reconfiguration of the edge set E . Let $u_{\text{prev}} = \mathcal{R}(H, i - 1)$ and $u_{\text{curr}} = \mathcal{R}(H, i)$. The transformation maps the old edge set E to a new edge set E' :

$$E' = (E \setminus \{(u_{\text{prev}}, u_{\text{curr}})\}) \cup \{(u_{\text{prev}}, u_{\text{new}}), (u_{\text{new}}, u_{\text{curr}})\}$$

To maintain structural integrity without losing reference addresses, the sequential update sequence is strictly ordered:

$$\begin{aligned} n_{u_{\text{new}}} &\leftarrow n_{u_{\text{prev}}} \\ n_{u_{\text{prev}}} &\leftarrow \text{Addr}(u_{\text{new}}) \end{aligned}$$

If $i = 0$ (insertion at the beginning), the head pointer itself is updated:

$$\begin{aligned} n_{u_{\text{new}}} &\leftarrow H \\ H &\leftarrow \text{Addr}(u_{\text{new}}) \end{aligned}$$

3 In-Place LinkedList Reversal: Loop Invariants and Correctness Proof

Let a linked list be represented by a sequential sequence of nodes $u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_N$. The task of in-place list reversal is to modify the successor pointers such that the directed path is inverted, resulting in $u_N \rightarrow u_{N-1} \rightarrow \dots \rightarrow u_1$, utilizing $O(1)$ auxiliary space.

The iterative algorithm maintains three pointer variables: p_t (prev), c_t (curr), and f_t (future), representing the previous node, current node, and next successor node at step $t \in \mathbb{N}_0$, respectively.

3.1 State Transition System

The initial state at $t = 0$ is defined as:

$$(p_0, c_0, f_0) = (\emptyset, H, n_H)$$

For each step $t \geq 1$, if $c_{t-1} \neq \emptyset$, the state transitions are defined by the following simultaneous updates:

$$\begin{aligned} f_t &= n_{c_{t-1}} \\ n_{c_{t-1}} &\leftarrow p_{t-1} \quad (\text{Pointer Inversion}) \\ p_t &= c_{t-1} \\ c_t &= f_t \end{aligned}$$

The termination condition is reached when $c_t = \emptyset$, at which point the new head pointer becomes p_t .

3.2 Proof of Correctness via Loop Invariants

Let the original list be denoted as the ordered sequence of addresses $A = (a_1, a_2, \dots, a_N)$, where $a_1 = H$. At any step t , the execution splits the list into two disconnected chains: a reversed prefix and an unreversed suffix.

Lemma 1 (Loop Invariant). At the start of step t (where $0 \leq t \leq N$), the following invariants hold:

- 1) The sublist starting at p_t forms a reversed chain: $a_t \rightarrow a_{t-1} \rightarrow \dots \rightarrow a_1 \rightarrow \emptyset$.
- 2) The sublist starting at c_t forms an intact unreversed chain: $a_{t+1} \rightarrow a_{t+2} \rightarrow \dots \rightarrow a_N \rightarrow \emptyset$.
- 3) The total node set is preserved, and no cycles are introduced.

Proof: We prove this by mathematical induction on t .

3.2.0.1 Base Case ($t = 0$): At step 0, $p_0 = \emptyset$, and $c_0 = a_1$. The reversed chain starting at p_0 is empty (vacuously true). The unreversed chain starting at c_0 is the entire original list $a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_N \rightarrow \emptyset$. The invariant holds.

3.2.0.2 Inductive Step: Assume the invariant holds at step $t - 1 < N$. The current state is $(p_{t-1}, c_{t-1}, f_{t-1}) = (a_{t-1}, a_t, a_{t+1})$. According to the transition system, we update:

$$f_t = n_{c_{t-1}} = n_{a_t} = a_{t+1}$$

We then execute the pointer inversion:

$$n_{a_t} \leftarrow p_{t-1} = a_{t-1}$$

This modifies the successor link of a_t to point to a_{t-1} . Since by inductive hypothesis $a_{t-1} \rightarrow a_{t-2} \rightarrow \dots \rightarrow a_1 \rightarrow \emptyset$ is already reversed, the new chain starting at a_t becomes:

$$a_t \rightarrow a_{t-1} \rightarrow a_{t-2} \rightarrow \dots \rightarrow a_1 \rightarrow \emptyset$$

We then update the state variables:

$$p_t = c_{t-1} = a_t$$

$$c_t = f_t = a_{t+1}$$

At the start of step t , the sublist starting at p_t is indeed $a_t \rightarrow a_{t-1} \rightarrow \dots \rightarrow a_1 \rightarrow \emptyset$, and the sublist starting at c_t is $a_{t+1} \rightarrow a_{t+2} \rightarrow \dots \rightarrow a_N \rightarrow \emptyset$. The inductive step is complete.

When $t = N$, we have $c_N = \emptyset$. By invariant (1), the sublist starting at p_N is the fully reversed list $a_N \rightarrow a_{N-1} \rightarrow \dots \rightarrow a_1 \rightarrow \emptyset$. The algorithm terminates, and returning p_N yields the correct reversed list. $\square$

4 Isomorphic Deep Copying of Lists with Random Pointers

We now address a more complex structural problem. Consider an augmented linked list where each node u contains both a sequential link n_u and a random link r_u , where r_u can point to any node within the list or to $\emptyset$. The objective is to construct an isomorphic deep copy of this graph, creating a brand new set of nodes with identical topology, such that:

$$\text{Addr}(u'_i) \neq \text{Addr}(u_j) \quad \forall i, j$$

$$d_{u'_i} = d_{u_i}, \quad n_{u'_i} = \text{Addr}(u'_{i+1}), \quad r_{u'_i} = \text{Addr}(u'_k) \text{ where } r_{u_i} = \text{Addr}(u_k)$$

4.1 Naive Hash-Map Approach

A straightforward solution uses a two-pass traversal and an auxiliary hash-map $\mathcal{M} : \mathcal{A} \rightarrow \mathcal{A}$ that maps original node addresses to their newly created counterparts.

- 1) Pass 1: Traverse the original list. For each node u_i , allocate u'_i , copy data $d_{u'_i} = d_{u_i}$, and store the mapping $\mathcal{M}[\text{Addr}(u_i)] = \text{Addr}(u'_i)$.
- 2) Pass 2: Traverse again. For each node u_i , set:

$$n_{u'_i} = \mathcal{M}[n_{u_i}]$$

$$r_{u'_i} = \mathcal{M}[r_{u_i}]$$

While simple, this approach has an auxiliary space complexity of $O(N)$ due to the storage requirements of the hash-map $\mathcal{M}$, which can be prohibitive in memory-constrained environments.

4.2 Optimal In-Place Interleaving Algorithm

To achieve $O(1)$ auxiliary space complexity, we can use an elegant three-phase interleaving technique. This algorithm avoids external maps by using the original list's successor pointers as temporary storage for the copy mappings.

4.2.1 Phase 1: Node Interleaving

We traverse the list and insert each copied node u'_i directly between u_i and its successor u_{i+1} . For each node u in the list:

```
allocate  $u'$  with  $d_{u'} = d_u$ 
 $n_{u'} \leftarrow n_u$ 
 $n_u \leftarrow \text{Addr}(u')$ 
```

This transforms the original chain $u_1 \rightarrow u_2 \rightarrow \dots$ into an interleaved chain:

$$u_1 \rightarrow u'_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_N \rightarrow u'_N \rightarrow \emptyset$$

4.2.2 Phase 2: Random Pointer Propagation

Because of the interleaved structure, the copy u' of any node u is always located at n_u . Consequently, the target of a copied random pointer $r_{u'}$ is simply the successor of the original random target r_u . For each original node u (processed by skipping over u' nodes):

$$n_u.r_u \leftarrow \begin{cases} \emptyset & \text{if } r_u = \emptyset \\ n_{r_u} & \text{if } r_u \neq \emptyset \end{cases}$$

Since n_{r_u} points directly to the copied node r'_u , this correctly sets all copy random pointers in a single linear pass.

4.2.3 Phase 3: Graph Decoupling

The final phase separates the interleaved list into two distinct, independent lists, restoring the original list's pointers and binding the copied list's pointers. Let $H_{\text{orig}} = H$ and $H_{\text{copy}} = n_H$. We traverse the interleaved list, updating the next links:

$$\begin{aligned} n_u &\leftarrow n_{n_u} \\ n_{u'} &\leftarrow n_{n_{u'}} \end{aligned}$$

This linear restoration restores the original graph topology while finalizing the copied graph.

4.3 Complexity Analysis

Let N be the number of nodes in the graph. * **Time Complexity***: Each of the three phases performs exactly one linear pass over the list. The total number of node visit operations is $N + N + 2N = 4N$, which is strictly $O(N)$. * **Space Complexity***: No external maps, tables, or stacks are allocated. The only allocations are the N required destination nodes. Thus, the auxiliary space complexity is strictly $O(1)$. This is spatially optimal.

5 Execution Semantics and Memory Topology

To understand the practical execution of these algorithms, we must model the underlying physical memory.

5.1 Program Stack and Heap Division

Memory is split into two primary segments:

- 1) The Stack: A LIFO (Last-In-First-Out) structure that manages function activation records (stack frames). Each frame contains local variables and return addresses.
- 2) The Heap: An unstructured pool of dynamic memory where nodes are allocated via explicit commands (e.g., new in C++ or JVM heap instantiation).

When a function like `swap(a, b)` or a linked list manipulator is executed, the reference values on the stack point to addresses on the heap.

5.2 Activation Records and Value Passing

Consider the following standard swap activation:

```
void swap(Node* a, Node* b) {
    Node* temp = a;
    a = b;
    b = temp;
}
```

In languages passing parameters by value (such as Java, or passing pointers by value in C), the pointers a and b inside the swap stack frame are local copies of the caller's pointers. Modifying them merely changes local stack variables:

$$\text{Stack}_{\text{swap}} = \{a \mapsto \text{Addr}(u_2), b \mapsto \text{Addr}(u_1), \text{temp} \mapsto \text{Addr}(u_1)\}$$

Once the activation record is popped off the stack upon function return, these changes are discarded, leaving the caller's pointers untouched. This formalizes why structural modification must mutate the heap-resident node links (e.g., n_u) rather than stack reference variables.

6 Conclusion

This paper has presented a rigorous, mathematically sound framework for node-link graphs and linear data structures. We verified the linear-time, in-place linked list reversal algorithm by establishing its loop invariants across discrete state transitions. Furthermore, we analyzed the deep-copy problem for lists with random pointers, proving that the three-phase interleaving technique achieves the theoretical lower bound of $O(1)$ auxiliary space while maintaining linear $O(N)$ time complexity. Finally, we modeled the stack and heap topologies, demonstrating the structural necessity of heap-level mutations in pointer-based algorithmic design.